JavaScript & TypeScript Cheat Sheet

Complete Reference Guide (Lessons 1–10)

For Automation Engineers | Interview Prep Ready

Lesson 1: Programming Fundamentals

Programming = Giving step-by-step instructions to a computer.

Every Program: Input → Process → Output

Key Mindset:

- Computer is fast but has zero common sense
- Always handle "what can go wrong?" (error handling)
- Manual test cases = programs in English

Lesson 2: Setup & First Code

Tools:

Tool	Purpose
Node.js	Runs JavaScript/TypeScript
VS Code	Code editor
npm	Package installer

Verify Commands:

```
node --version
npm --version
```

First Code:

```
console.log("Hello World!");
```

- console.log() → prints to screen
- Every line ends with ;
- Code is CASE-SENSITIVE

Lesson 3: Variables

Definition: A named box that stores data.

3 Keywords:

```
const url = "site.com";  // Never changes
let count = 0;           // Can change
var old = "x";           // ⚠ Never use!
```

Keyword	Changes?	Use For
const	No	URLs, fixed values

let	Yes	Counters, status
var	Yes	Don't use!

4 Data Types:

```
let name = "John";    // String (quotes)
let age = 25;         // Number (no quotes)
let isActive = true;  // Boolean (true/false)
let empty = null;     // Null (no value)
```

Naming Rule (camelCase):

```
let firstName = "John";       // ✅
let loginPageUrl = "...";     // ✅
let first name = "John";      // ❌No spaces!
```

Lesson 4: Operators

Arithmetic:

```
+   // Addition       5 + 3 = 8
-   // Subtraction    10 - 4 = 6
*   // Multiplication 3 * 4 = 12
/   // Division       10 / 2 = 5
%   // Remainder      10 % 3 = 1
```

Template Literals (use backticks!):

```
let name = "John";
console.log(`Hello ${name}!`);  // ✅backticks
// NOT '${name}' or "${name}" — only backticks work!
```

Comparison:

```
===  // Exactly equal (USE THIS!)
!==  // Not equal
>    // Greater than
<    // Less than
>=   // Greater or equal
<=   // Less or equal
```

Logical:

```
&&   // AND (all must be true)
||   // OR (one must be true)
!    // NOT (flips true/false)
```

Critical Rule:

```
=    // Assign: let x = 5
==   // Loose compare: 5 == "5" → true  ⚠ avoid!
===  // Strict compare: 5 === "5" → false ✓ always use!
```

Lesson 5: Conditional Statements

Simple if:

```
if (isLoggedIn) {
  console.log("Welcome!");
}
```

if/else:

```
if (passed) {
  console.log("PASS");
} else {
  console.log("FAIL");
}
```

if/else if/else:

```
if (type === "gold") {
  discount = 20;
} else if (type === "silver") {
  discount = 10;
} else {
  discount = 0;
}
```

Ternary (one-line):

```
let status = passed ? "PASS" : "FAIL";
// condition ? ifTrue : ifFalse
```

Rules:

- Checks top to bottom, STOPS at first match
- Always add else safety net
- Use === not =

Lesson 6: Loops

for loop (known count):

```
for (let i = 1; i <= 5; i++) {
```

```
  console.log(i);
}
// i++ adds 1 each time
// i-- subtracts 1 each time
```

while loop (unknown count):

```
let attempts = 3;
while (attempts > 0) {
  attempts--;
}
```

do...while (runs at least once):

```
do {
  console.log("Runs once minimum");
} while (condition);
```

for...of (loop through list):

```
let browsers = ["Chrome", "Firefox"];
for (let browser of browsers) {
  console.log(browser);
}
```

Loop Control:

```
break;     // EXIT loop immediately
continue;  // SKIP this round, go to next
```

⚠ Avoid Infinite Loops:

```
// Always ensure loop has an exit!
while (i <= 5) {
  i++;  // This makes it stop eventually!
}
```

Lesson 7: Arrays

Definition: Stores multiple values in one variable.

Create & Access:

```
let browsers = ["Chrome", "Firefox", "Safari"];
//              index 0    index 1     index 2
browsers[0];  // Chrome (starts at 0!)
browsers[2];  // Safari
```

Useful Methods:

```
browsers.length          // 3 (count items)
browsers.push("Edge")    // Add to end
browsers.pop()           // Remove from end
browsers.includes("Chrome")  // true (check exists)
browsers.indexOf("Firefox")  // 1 (find position)
```

Pro Trick — Get Last Item:

```
browsers[browsers.length - 1]  // Always last item!
```

Arrays + Loops:

```
for (let browser of browsers) {
  console.log(browser);
}
```

Lesson 8: Functions

Definition: Reusable block of code. Write once, use anywhere.

Simple Function:

```
function showWelcome() {
  console.log("Welcome!");
}
showWelcome();  // Must CALL to run!
```

With Parameters:

```
function greet(name) {  // name = parameter
  console.log(`Hi ${name}`);
}
greet("John");  // "John" = argument
```

With Return:

```
function add(a, b) {
  return a + b;  // Sends result back
}
let sum = add(5, 3);  // sum = 8
```

Term	Meaning
Parameter	Placeholder in definition
Argument	Real value when calling

return	Sends result back + stops function

Lesson 9: Objects

Definition: Groups related data together.

Create & Access:

```
let user = {
  username: "john",   // key: value
  password: "pass123",
  isActive: true
};
user.username;  // john (dot notation)
```

Update & Add:

```
user.isActive = false;  // Update existing
user.role = "admin";    // Add new property
```

Term	Meaning
Key	The label (username)
Value	The data ("john")

Array of Objects (Important!):

```
let users = [
  { username: "admin", role: "admin" },
  { username: "tester", role: "tester" }
];
for (let user of users) {
  console.log(user.username);
}
```

Lesson 10: TypeScript

Definition: JavaScript + Type Safety. Catches errors before code runs.

File extension: .ts (not .js)

Type Annotations:

```
let name: string = "John";
let age: number = 25;
let isActive: boolean = true;
```

TypeScript Catches Errors:

```
let price: number = 5000;
price = "hello";  // ❌ERROR! Not a number!
```

Functions With Types:

```
function add(a: number, b: number): number {
  return a + b;
}
// params have types ↑    return type ↑
```

Interface (object blueprint):

```
interface User {
  username: string;
  password: string;
  isActive: boolean;
}
let user: User = {
  username: "john",
  password: "pass123",
  isActive: true
};
```

Array of Typed Objects:

```
let users: User[] = [
  { username: "admin", password: "x", isActive: true },
  { username: "tester", password: "y", isActive: false }
];
// User[] = array of User objects
```

Type Arrays:

```
let names: string[] = ["John", "Priya"];
let prices: number[] = [100, 200];
let flags: boolean[] = [true, false];
```

Interview Quick Answers

Q: const vs let?
const = value never changes. let = value can change.

Q: == vs ===?
== compares value only (loose). === compares value AND type (strict). Always use ===.

Q: What is an array?
A variable that stores multiple values. Index starts at 0.

Q: What is a function?
Reusable block of code. Write once, call anywhere.

Q: Parameter vs Argument?
Parameter = placeholder in definition. Argument = real value when calling.

Q: What is an object?
Groups related data as key-value pairs.

Q: JavaScript vs TypeScript?
TypeScript = JavaScript + type safety. Catches type errors before runtime.

Q: What is an interface?
A blueprint defining the structure an object must follow.

Q: What is a loop?
Repeats code multiple times automatically.

Q: break vs continue?
break = exit loop. continue = skip current round, continue loop.

Top 10 Golden Rules

1. const = never changes | let = can change | never var
2. Always use === not ==
3. Backticks ` ` for template literals with ${}
4. Always declare variables with let/const
5. Always add else as safety net
6. Always ensure loops have an exit
7. camelCase for variable names
8. Arrays start at index 0
9. Functions: define AND call
10. TypeScript catches errors before runtime

Top Common Mistakes

1. = vs === in conditions

```
if (x = 5)    // ✕assigns
```

```
if (x === 5)  // ✓compares
```

2. Quotes for template literals

```
'Hi ${name}'   // ✗broken
`Hi ${name}`   // ✓works
```

3. Array index

```
arr[1]  // ✗second item
arr[0]  // ✓first item
```

4. Function not called

```
function go(){}  // ✗never runs
go();            // ✓runs
```

5. Object syntax

```
{ name = "x" }  // ✗wrong
{ name: "x" }   // ✓correct
```

Playwright + TypeScript Cheat Sheet

🎭 Playwright + TypeScript — Complete Interview Cheat Sheet

Abiram's 2-Month Learning Journey | Lessons 1–35 | Scan in 15 mins before any interview

---

⚡ ARCHITECTURE WARMUP — Playwright vs Selenium

```
SELENIUM (Old way — slow):
Your Test Code
    ↓  HTTP Request (slow round-trip per action!)
WebDriver Server
    ↓  HTTP
Browser
→ Every single action = a new HTTP round-trip = SLOW 🐌

PLAYWRIGHT (Modern way — fast):
Your Test Code
    ↓  WebSocket (single persistent connection!)
Browser (Chromium / Firefox / WebKit)
→ One connection stays open the entire test = FAST ⚡
→ No extra server needed!
→ Auto-waiting built in — no flakiness!

KEY DIFFERENCE:
Selenium → HTTP request per action → slow, needs WebDriver
Playwright → WebSocket tunnel → persistent, fast, auto-waits
```

> 🎤 ***Interview Phrase:*** *"Playwright uses a single persistent WebSocket connection to the browser, unlike Selenium which makes a new HTTP request for every action. This is why Playwright is faster and has built-in auto-waiting that eliminates most*

> *flakiness."*

📋 LESSONS AT A GLANCE

Phase	Lessons	Topic
Phase 1	1–6	Programming Foundations (Variables, Loops, Conditionals)
Phase 2	7–14	TypeScript (Arrays, Objects, Functions, OOP, Async/Await)
Phase 3	15–22	Playwright Fundamentals (Locators, Actions, Assertions, Waits)
Phase 4	23–29	Advanced Playwright (POM, Fixtures, API, Visual, Parallel)
Phase 5	30–35	Framework Pro (Design, Config, Env, CI/CD, Best Practices)

🔧 MODULE 1 — Setup & Configuration (`playwright.config.ts`)

Core Concept: The single source of truth for your entire framework — browsers, timeouts, environments, reporters all live here.

```
// playwright.config.ts — Production-grade setup
import { defineConfig, devices } from '@playwright/test';

const ENV = process.env.ENV || 'qa';

const envConfig = {
  dev:  { baseURL: 'https://dev.myapp.com',  timeout: 60000 },
```

```
  qa:   { baseURL: 'https://qa.myapp.com',   timeout: 30000 },
  prod: { baseURL: 'https://myapp.com',       timeout: 20000 },
};

export default defineConfig({
  testDir: './tests',
  workers: 4,
  retries: 1,
  timeout: envConfig[ENV].timeout,

  reporter: [
    ['html', { open: 'on-failure' }],
    ['junit', { outputFile: 'test-results.xml' }],  // For Jenkins/CI!
    ['list'],
  ],

  use: {
    baseURL:    envConfig[ENV].baseURL,
    screenshot: 'only-on-failure',
    video:      'retain-on-failure',
    trace:      'on-first-retry',
  },

  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox',  use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit',   use: { ...devices['Desktop Safari'] } },
  ],
});
```

🎤 **Interview Prep:** - *"What does* `retries: 1` *do?"* → Reruns a failed test once before marking it as failed — reduces flakiness from transient network issues. - *"What is* `trace: 'on-first-retry'`*?"* → Records a full step-by-step trace (screenshots + network + console) when a test is retried, viewable via `npx playwright show-trace`.

⚠ **Common Pitfall:** Using `timeout` inside `use:{}` sets the **action timeout** (per-action). The top-level `timeout` sets the **test timeout** (whole test). These are different!

🔍 MODULE 2 — Modern Locators (Lesson 18)

Core Concept: Locators tell Playwright *where* to find an element. Modern semantic locators are more resilient than CSS/XPath because they reflect how users see the page.

```
        // ✅ PREFERRED — Semantic locators (Playwright recommended order):

        // 1. By Role (most preferred!)
        await page.getByRole('button', { name: 'Submit' }).click();
        await page.getByRole('textbox', { name: 'Username' }).fill('student');
        await page.getByRole('link', { name: 'Get Started' }).click();
        await page.getByRole('heading', { name: 'Dashboard' });

        // 2. By Label (for form fields)
        await page.getByLabel('Email Address').fill('test@test.com');

        // 3. By Placeholder
        await page.getByPlaceholder('Enter your email').fill('test@test.com');

        // 4. By Text
        await page.getByText('Submit Form').click();

        // 5. By ID (stable but not always present)
        await page.locator('#username').fill('student');

        // ❌ AVOID — fragile selectors:
        await page.locator('div > div > input').fill('student'); // breaks on
restructure
        await page.locator('.btn').click();                      // too generi
```

Priority Order: | Priority | Locator | Why | |—|—|—| | 1st ⭐ | `getByRole()` | Mirrors how users see the page | | 2nd ⭐ | `getByLabel()` | Tied to form labels — stable | | 3rd ⭐ | `getByPlaceholder()` | For inputs with hints | | 4th ⭐ | `getByText()` | When text is unique | | 5th | `#id` | Stable if IDs don't change | | Last ⚠️ | `.class` / XPath | Fragile — avoid |

🎤 **Interview Prep:** - *"Why prefer* `getByRole` *over CSS selectors?"* → Role-based locators are semantic — they target how the element is perceived by users and assistive tech, making them resilient to CSS/HTML changes. - *"What is the difference between* `getByText()` *and* `getByRole()`*?"* → `getByText` matches visible text on any element; `getByRole` matches elements by their ARIA role plus optional name, which is more specific and accessible.

⚠️ **Common Pitfall:** `page.locator('.card')` matches ALL elements with that class. Use

`.nth(0)` or a more specific locator to target one.

✅ MODULE 3 — Assertions & Soft Assertions (Lesson 20)

Core Concept: Assertions verify the test actually passed. Without them, your test is just clicking — not verifying.

```
// PAGE ASSERTIONS:
await expect(page).toHaveURL('/dashboard');
await expect(page).toHaveURL(/dashboard/);        // Regex match!
await expect(page).toHaveTitle(/ShopEasy/);

// ELEMENT ASSERTIONS:
await expect(page.locator('#msg')).toBeVisible();
await expect(page.locator('#msg')).not.toBeVisible(); // .not reverses
await expect(page.locator('#msg')).toHaveText('Login successful!');
await expect(page.locator('#msg')).toContainText('successful'); // part
await expect(page.locator('#btn')).toBeEnabled();
await expect(page.locator('#btn')).toBeDisabled();
await expect(page.locator('#chk')).toBeChecked();
await expect(page.locator('#input')).toHaveValue('student');
await expect(page.locator('.card')).toHaveCount(5); // count elements

// SOFT ASSERTIONS — test continues even if this fails:
await expect.soft(page.locator('#name')).toHaveText('John');
await expect.soft(page.locator('#email')).toHaveText('john@test.com');
// Test keeps running — collects ALL failures at end!

// Helpful failure message:
await expect(page.locator('.success'))
  .toHaveText('Login successful!', { timeout: 5000 });
```

🎤 **Interview Prep:** - *"What is the difference between `toHaveText()` and `toContainText()`?"* → `toHaveText` requires an exact full match; `toContainText` only requires the element to include that substring anywhere in its text. - *"What are soft assertions and when would you use them?"* → Soft assertions use `expect.soft()`. The test continues running even if a soft assertion fails, collecting all failures and reporting

them together. Useful for form validation testing where you want to verify multiple fields in one run.

⚠ **Common Pitfall:** Forgetting `await` on assertions — `expect(...)` is async in Playwright. Without `await`, the assertion doesn't actually wait or execute properly.

⏱ MODULE 4 — Auto-Waiting & Manual Waits (Lesson 21)

Core Concept: Playwright auto-waits for elements before every action (checks visible, enabled, stable). Manual waits handle special timing scenarios.

```
        // AUTO-WAITING (happens behind the scenes for every action!):
        await page.click('#submit');
        // Playwright automatically waits for:
        // ✅ Element exists in DOM
        // ✅ Element is visible
        // ✅ Element is enabled
        // ✅ Element is not animating
        // THEN clicks!

        // MANUAL WAITS — only when auto-wait isn't enough:

        // 1. Wait for URL to change (after login/navigation):
        await page.waitForURL(/dashboard/);

        // 2. Wait for element to appear:
        await page.waitForSelector('#successMessage');

        // 3. Wait for locator state:
        await page.locator('#loader').waitFor({ state: 'hidden' }); // spinner
gone!
        await page.locator('#dashboard').waitFor({ state: 'visible' });

        // 4. Wait for page load state:
        await page.waitForLoadState('networkidle');    // No network for 500ms
        await page.waitForLoadState('domcontentloaded'); // HTML ready

        // ❌ AVOID — hard-coded waits:
```

```
        await page.waitForTimeout(3000);  // Always slow, never reliable!
```

🎤 **Interview Prep:** - *"How does Playwright's auto-waiting work?"* → Before every action like click or fill, Playwright automatically checks that the element is attached to the DOM, visible, enabled, and not animating. This eliminates most timing-related flakiness without writing extra wait code. - *"When would you use* `waitForLoadState('networkidle')`*?"* → After navigation or form submissions that trigger API calls — it waits until there are no network requests for 500ms, ensuring all dynamic content has loaded.

⚠ **Common Pitfall:** Over-using `waitForTimeout()` — it always waits the full time even if the page is ready in 200ms. This makes your test suite unnecessarily slow.

🪟 MODULE 5 — Advanced UI: Tabs, iFrames, Alerts (Lesson 22)

Core Concept: Real apps have browser elements that live *outside* the normal page — these require special handling.

```
        // 1. BROWSER ALERTS (dialogs):
        // Set listener BEFORE the action that triggers it!
        page.on('dialog', async (dialog) => {
          console.log(dialog.message()); // get message text
          await dialog.accept();         // click OK
          // await dialog.dismiss();     // click Cancel
        });
        await page.click('#deleteBtn');  // triggers alert → listener handles i

        // 2. NEW TABS:
        const [newPage] = await Promise.all([
          page.context().waitForEvent('page'), // listen for new tab
          page.getByRole('link', { name: 'Open' }).click() // triggers new tab
        ]);
        await newPage.waitForLoadState();
        await expect(newPage.getByRole('heading', { name: 'New Window'
})).toBeVisible();
        await newPage.close(); // back to original page

        // 3. iFRAMES:
```

```
        // First get a handle to the frame:
        const frame = page.frameLocator('#payment-iframe');
        // Then interact inside the frame:
        await frame.getByLabel('Card Number').fill('4111111111111111');
        await frame.getByRole('button', { name: 'Pay' }).click();

        // 4. FILE UPLOAD:
        await page.locator('#fileUpload').setInputFiles('path/to/file.pdf');
```

🎤 **Interview Prep:** - *"How do you handle iframes in Playwright?"* → Use `page.frameLocator('#frame-id')` to get a frame handle, then chain locators inside it just like the main page. Playwright's frame locators auto-wait just like page locators. - *"Why must you set the dialog listener before triggering the action?"* → If the alert fires before the listener is attached, Playwright auto-dismisses it and your test misses it. The listener must be registered first.

⚠ **Common Pitfall:** Trying to use `page.locator()` on elements inside an iframe — it won't find them. You must use `page.frameLocator()` first.

🧬 MODULE 6 — Test Fixtures (Lesson 23)

Core Concept: Fixtures are reusable setup/teardown helpers injected into tests. They eliminate repetitive `beforeEach` login code.

```
        // fixtures.ts — Define custom fixtures
        import { test as base, Page } from '@playwright/test';

        type MyFixtures = {
          loggedInPage: Page;
        };

        export const test = base.extend<MyFixtures>({

          loggedInPage: async ({ page }, use) => {
            // SETUP: runs before every test
            await page.goto('https://practicetestautomation.com/practice-test-
login/');
            await page.locator('#username').fill('student');
            await page.locator('#password').fill('Password123');
            await page.getByRole('button', { name: 'Submit' }).click();
```

```
    await page.waitForURL(/logged-in-successfully/);

    await use(page); // ← TEST RUNS HERE (baton handoff!)

    // TEARDOWN: runs after every test
    console.log('✅ Test complete — cleaning up!');
  }
});

export { expect } from '@playwright/test';

// ---------------------------------------------------
// login.spec.ts — Using the fixture:
import { test, expect } from '../fixtures'; // ← YOUR fixtures, not Playwright's!

test('Dashboard loads after login', async ({ loggedInPage }) => {
  await expect(loggedInPage.getByText('Logged In Successfully')).toBeVisible();
});

test('Logout link is visible', async ({ loggedInPage }) => {
  await expect(loggedInPage.getByRole('link', { name: 'Log out' })).toBeVisible();
});
// No login code in either test! Fixture handles it! ✅
```

🎤 **Interview Prep:** - *"What is a Playwright fixture and how is it different from `beforeEach`?"* → Fixtures are dependency-injected, composable, and lazily initialized — only created if the test requests them. `beforeEach` always runs for every test in the block. Fixtures can also be shared across files and chained together. - *"What does `await use(page)` mean inside a fixture?"* → It's the handoff point — everything before `use()` is setup, everything after is teardown. The test body runs during the `await use()` call.

⚠ **Common Pitfall:** Importing `test` from `@playwright/test` instead of your `./fixtures` file — your custom fixtures won't be available!

🏛 MODULE 7 — Page Object Model — POM (Lesson 24)

Core Concept: POM separates *what to test* (test files) from *how to interact* (page objects). When a locator changes, you fix it in ONE place.

```
        // pages/LoginPage.ts — The Page Object
        import { Page, Locator } from '@playwright/test';

        export class LoginPage {
          private page: Page;

          // Locators as properties — fix once, works everywhere!
          private usernameField: Locator;
          private passwordField: Locator;
          private submitButton: Locator;
          readonly errorMessage: Locator;

          constructor(page: Page) {
            this.page = page;
            this.usernameField = page.locator('#username');
            this.passwordField = page.locator('#password');
            this.submitButton  = page.getByRole('button', { name: 'Submit' });
            this.errorMessage  = page.locator('#error');
          }

          async navigateTo() {
            await this.page.goto('/practice-test-login/'); // uses baseURL from
config!
          }

          async loginAs(username: string, password: string) {
            await this.usernameField.fill(username);
            await this.passwordField.fill(password);
            await this.submitButton.click();
          }
        }

        // ------------------------------------------------
        // tests/login.spec.ts — Clean test using POM
        import { test, expect } from '@playwright/test';
        import { LoginPage } from '../pages/LoginPage';
```

```
        test('Valid login redirects to dashboard', async ({ page }) => {
          const loginPage = new LoginPage(page);
          await loginPage.navigateTo();
          await loginPage.loginAs('student', 'Password123');
          await expect(page).toHaveURL(/logged-in-successfully/);
        });
        // Test reads like plain English! No locators visible! ✅
```

Project structure:

```
playwright-automation/
├── pages/
│   ├── LoginPage.ts
│   ├── DashboardPage.ts
│   └── CheckoutPage.ts
├── tests/
│   ├── login.spec.ts
│   └── checkout.spec.ts
├── fixtures.ts
├── playwright.config.ts
└── .env
```

🎤 **Interview Prep:** - *"What is the Page Object Model and why do you use it?"* → POM is a design pattern where each page/component of the app has its own class containing locators and methods. It separates test logic from page interaction logic, making maintenance easier — when a locator changes, you update only the page class, not every test. - *"What is the difference between POM and fixtures?"* → POM is about *organizing locators and page interactions* into classes. Fixtures handle *test setup/teardown*. They complement each other — a fixture can instantiate a POM class after performing login setup.

⚠ **Common Pitfall:** Putting assertions inside the Page Object. POMs should only contain navigation and interactions — keep assertions in the test file where intent is clear.

📊 MODULE 8 — Data-Driven Testing (Lesson 25)

Core Concept: One test template + many data sets = many test scenarios without code duplication.

```
        // tests/login-ddt.spec.ts
```

```
        import { test, expect } from '@playwright/test';
        import { LoginPage } from '../pages/LoginPage';

        // All test data in one place:
        const loginTestData = [
          { scenario: 'Valid credentials',  username: 'student',   password:
'Password123', expected: 'pass' },
          { scenario: 'Invalid username',   username: 'wrongUser', password:
'Password123', expected: 'fail' },
          { scenario: 'Invalid password',   username: 'student',   password:
'wrongPass',   expected: 'fail' },
          { scenario: 'Empty fields',       username: '',          password: ''
expected: 'fail' },
        ];

        // ONE loop creates MANY tests:
        for (const data of loginTestData) {
          test(`Login: ${data.scenario}`, async ({ page }) => {
            const loginPage = new LoginPage(page);
            await loginPage.navigateTo();
            await loginPage.loginAs(data.username, data.password);

            if (data.expectedResult === 'pass') {
              await expect(page).toHaveURL(/logged-in-successfully/);
            } else {
              await expect(loginPage.errorMessage).toBeVisible();
            }
          });
        }
        // 4 test cases from 4 rows of data! Add more rows → more tests! ✓
```

🎤 **Interview Prep:** - *"How do you implement data-driven testing in Playwright?"* → Store test data in an array of objects, then use a `for...of` loop to dynamically generate `test()` blocks. Each iteration becomes a separate named test in the report. - *"What is `test.each()` and when would you use it?"* → `test.each()` is Playwright's built-in data-driven method, useful for simpler tabular data. For complex objects with multiple properties, a `for...of` loop with an object array is more readable.

⚠ **Common Pitfall:** Defining the test data inside the test function — it creates one test run, not multiple. The `for` loop must be *outside* the `test()` call.

🌐 MODULE 9 — API Testing & Mocking (Lesson 26)

Core Concept: Playwright's `request` fixture sends HTTP calls directly to the server — no browser needed. 10x faster than UI tests.

```
// tests/api.spec.ts
import { test, expect, request } from '@playwright/test';

// GET Request:
test('GET — Fetch user details', async ({ request }) => {
  const response = await request.get('https://reqres.in/api/users/2');
  expect(response.status()).toBe(200);

  const body = await response.json();
  expect(body.data.id).toBe(2);
  expect(body.data.email).toBe('janet.weaver@reqres.in');
});

// POST Request:
test('POST — Create new user', async ({ request }) => {
  const response = await request.post('https://reqres.in/api/users', {
    data: { name: 'Abiram', job: 'Automation Engineer' }
  });
  expect(response.status()).toBe(201);
  const body = await response.json();
  expect(body.name).toBe('Abiram');
});

// API MOCKING / INTERCEPTING — stub an API response:
test('Mock API Response', async ({ page }) => {
  // Intercept the API call and return fake data:
  await page.route('**/api/users', route => {
    route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify({ data: [{ id: 99, name: 'Mock User' }] })
    });
  });

  await page.goto('/users');
```

```
  await expect(page.getByText('Mock User')).toBeVisible();
  // UI shows our fake data — real API never called! ✅
});
```

Status Code Quick Reference: | Code | Meaning | |—|—| | `200` | OK — Success | | `201` | Created — POST success | | `400` | Bad Request — wrong data sent | | `401` | Unauthorized — not logged in | | `404` | Not Found | | `500` | Server Error |

🎤 **Interview Prep:** - *"What is the difference between API testing and UI testing in Playwright?"* → API testing uses the `request` fixture and sends HTTP calls directly to the server without opening a browser. It's ~10x faster and tests business logic. UI testing validates the frontend. Both are needed in a complete framework. - *"What is network mocking and when is it useful?"* → Using `page.route()` to intercept API calls and return controlled fake responses. Useful for testing UI behavior when the backend is unavailable, or testing error states like 500 responses.

⚠ **Common Pitfall:** Using `{ request }` from `@playwright/test` directly — this creates an isolated API context. If you want cookies/auth from the page session, use `page.request` instead.

🔀 MODULE 10 — Parallel Execution & Test Tagging (Lesson 28)

Core Concept: Workers run tests simultaneously — reducing total suite time dramatically. Tags let you run targeted subsets.

```
// playwright.config.ts — control workers:
export default defineConfig({
  workers: 4,        // 4 tests run at same time
  // workers: '50%' // use half of CPU cores
  // workers: 1     // sequential (use for debugging!)
});

// Command line overrides:
// npx playwright test --workers=4
// npx playwright test --workers=1

// Parallel block (all tests inside run simultaneously):
test.describe.parallel('Login Tests', () => {
  test('Valid login', async ({ page }) => { });
```

```
  test('Invalid login', async ({ page }) => { });
});

// Serial block (tests run in ORDER — for dependent flows):
test.describe.serial('Checkout Flow', () => {
  test('Step 1: Add to cart', async ({ page }) => { });
  test('Step 2: Enter address', async ({ page }) => { });
  test('Step 3: Pay', async ({ page }) => { });
});

// TAGGING — run only specific tests:
test('Login test @smoke', async ({ page }) => { });
test('Payment test @regression', async ({ page }) => { });
test('Profile test @smoke @regression', async ({ page }) => { });

// Run only tagged tests:
// npx playwright test --grep @smoke
// npx playwright test --grep @regression
// npx playwright test --grep-invert @slow  (exclude slow tests)

// Run a specific project (browser):
// npx playwright test --project=chromium
// npx playwright test --project=firefox
```

🎤 **Interview Prep:** - *"What are Playwright workers?"* → Workers are separate processes that run tests in parallel. With 4 workers, 4 tests run simultaneously, cutting suite time by ~75%. - *"When would you use* `test.describe.serial`*?"* → When tests have a dependency chain — e.g., a shopping flow where paying depends on having items in cart. Serial ensures they run in order and stops on the first failure.

⚠ **Common Pitfall:** Shared state between parallel tests — each worker runs in an isolated browser context, but if tests write to the same file or database record, they can interfere. Keep tests independent.

🏗 MODULE 11 — Framework Design & Structure (Lesson 30)

Core Concept: A scalable framework is organized so any engineer can find, run, and extend it without asking questions.

```
playwright-automation/              ← Root
│
├── 📁 .github/
│   └── workflows/playwright.yml    ← CI/CD pipeline
│
├── 📁 pages/                       ← Page Objects (POM)
│   ├── LoginPage.ts
│   ├── DashboardPage.ts
│   └── CheckoutPage.ts
│
├── 📁 tests/                       ← Test files
│   ├── login.spec.ts
│   ├── checkout.spec.ts
│   └── api.spec.ts
│
├── 📁 test-data/                   ← Data-driven test data
│   └── loginData.ts
│
├── 📁 utils/                       ← Shared helpers
│   └── helpers.ts
│
├── 📁 config/                      ← Environment configs
│   └── env.config.ts
│
├── fixtures.ts                     ← Custom fixtures
├── playwright.config.ts            ← Master config
├── .env                            ← Local secrets (never commit!)
├── .gitignore                      ← Excludes .env, node_modules
└── package.json
```

🎤 **Interview Prep:** - *"How would you design a Playwright framework from scratch?"* → Start with POM for page interactions, fixtures for shared setup, a config file for environment URLs, test data separated from test logic, and GitHub Actions for CI/CD. Use TypeScript throughout for type safety.

⚠ **Common Pitfall:** Putting all tests in one file — makes it impossible to run targeted subsets and slows down the team.

🌍 MODULE 12 — Environment Handling (Lessons 31–

32)

Core Concept: One framework should run against any environment (DEV/QA/PROD) by passing a single variable — never by changing code.

```
        // config/env.config.ts
        const ENV = process.env.ENV || 'qa';

        export const envConfig = {
          dev:  { baseURL: 'https://dev.myapp.com',  username: process.env.DEV_
},
          qa:   { baseURL: 'https://qa.myapp.com',   username: process.env.QA_U
},
          prod: { baseURL: 'https://myapp.com',       username:
process.env.PROD_USER },
        };

        export const config = envConfig[ENV];

        // .env file (NEVER commit to GitHub!):
        // QA_USERNAME=student
        // QA_PASSWORD=Password123

        // Running on different environments:
        // Windows: $env:ENV="dev"  → npx playwright test
        // Mac/Linux: ENV=qa npx playwright test
        // CI/CD: Set ENV in GitHub Actions env: section
```

🎤 **Interview Prep:** - *"How do you manage test credentials securely?"* → Store them in a `.env` file locally (listed in `.gitignore` so it's never committed). In CI/CD, use GitHub Secrets — they're encrypted and injected as environment variables at runtime. - *"How do you run tests on different environments?"* → Set an `ENV` environment variable before running (`ENV=prod npx playwright test`). The config reads this variable and picks the right base URL and credentials.

⚠ **Common Pitfall:** Hard-coding URLs or credentials directly in test files. When the environment changes, you have to update dozens of files.

🚀 MODULE 13 — CI/CD with GitHub Actions (Lesson

33)

Core Concept: Every code push automatically triggers your full Playwright suite on a fresh Linux machine — zero manual effort.

```yaml
# .github/workflows/playwright.yml

name: Playwright Tests 🎭

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '30 3 * * 1-5'    # 9 AM IST weekdays (3:30 AM UTC)

jobs:
  playwright-tests:
    runs-on: ubuntu-latest
    timeout-minutes: 60

    steps:
      - name: 📥 Checkout code
        uses: actions/checkout@v4

      - name: 🟢 Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'              # Cache node_modules for speed!

      - name: 📦 Install dependencies
        run: npm ci                 # Faster + locked versions

      - name: 🎭 Install Playwright browsers
        run: npx playwright install --with-deps chromium

      - name: 🧪 Run tests
        run: npx playwright test --project=chromium
```

```
                env:
                  ENV: qa
                  QA_USERNAME: ${{ secrets.QA_USERNAME }}    # From GitHub Secre
                  QA_PASSWORD: ${{ secrets.QA_PASSWORD }}

              - name: 📊 Upload report
                uses: actions/upload-artifact@v4
                if: always()                                 # Upload even on
failure!
                with:
                  name: playwright-report-${{ github.run_number }}
                  path: playwright-report/
                  retention-days: 30
```

Flow:

```
You push code
    ↓
GitHub detects push on main
    ↓
Fresh Ubuntu machine created
    ↓
Steps run: checkout → node → npm ci → browsers → tests
    ↓
All pass? → ✅ Green checkmark on commit
Any fail? → ❌ Red X → team notified → DON'T merge!
    ↓
Report uploaded as artifact → download & view HTML report
```

🎤 **Interview Prep:** - *"Why* `npm ci` *instead of* `npm install` *in CI?"* → `npm ci` uses the exact versions from `package-lock.json`, is faster, and fails if the lock file is out of sync — ensures reproducibility across environments. - *"Why* `--with-deps` *on* `playwright install`*?"* → Linux CI machines don't have browser system libraries pre-installed (like `libglib`, `libnss`). `--with-deps` installs these OS-level dependencies. Without it, browsers fail to launch in CI even though they work locally on Windows.

⚠️ **Common Pitfall:** Never put passwords in the workflow YAML file — they're visible to everyone with repo access. Always use `${{ secrets.MY_SECRET }}`.

📸 MODULE 14 — Visual Testing (Lesson 27)

Core Concept: Screenshot comparison catches UI regressions (broken layouts, color changes, moved elements) that normal assertions miss.

```
// First run — creates baseline screenshot:
// npx playwright test visual.spec.ts --update-snapshots

test('Login page visual check', async ({ page }) => {
  await page.goto('/practice-test-login/');
  await page.waitForLoadState('networkidle');

  // Full page screenshot comparison:
  await expect(page).toHaveScreenshot('login-page.png');

  // Element-level screenshot:
  await expect(page.locator('form')).toHaveScreenshot('login-form.png')

  // Allow small pixel differences (font rendering, animations):
  await expect(page).toHaveScreenshot('dashboard.png', {
    maxDiffPixelRatio: 0.01  // Allow up to 1% difference
  });
});

// Update baseline when UI intentionally changes:
// npx playwright test --update-snapshots
```

🎤 **Interview Prep:** - *"When would you use visual testing vs assertion-based testing?"* → Assertion-based testing verifies data and behavior. Visual testing catches layout shifts, color changes, and UI regressions that can't be expressed as text assertions. Use both — assertions for correctness, screenshots for visual integrity.

⚠ **Common Pitfall:** Running baseline on one OS and comparing on another (e.g., Windows baseline → Linux CI). Screenshots differ slightly between OSes due to font rendering. Generate baselines in CI too.

📊 MODULE 15 — Reporting (Lesson 29)

Core Concept: Reports translate test results into evidence — for managers, developers,

and CI pipelines.

```
// playwright.config.ts — multiple reporters:
reporter: [
  ['html', { open: 'on-failure' }],              // Beautiful browser vi
  ['junit', { outputFile: 'test-results.xml' }], // Jenkins/CI reads th
  ['json', { outputFile: 'test-results.json' }], // Machine-readable
  ['list'],                                        // Live terminal outp
],

use: {
  screenshot: 'only-on-failure',    // Screenshot on fail
  video:      'retain-on-failure',  // Video replay on fail
  trace:      'on-first-retry',     // Full step trace on retry
},
```

```
# View HTML report:
npx playwright show-report

# View trace for a failed test:
npx playwright show-trace trace.zip
# Shows every action, screenshot, network call, console log!
```

🎤 **Interview Prep:** - *"What is a Playwright trace and when is it useful?"* → A trace is a complete recording of the test — every action, DOM snapshot, network request, and console log. View it with `npx playwright show-trace`. Essential for debugging flaky tests in CI where you can't observe the browser live.

🏆 MODULE 16 — Best Practices (Lesson 35)

Core Concept: These habits separate a junior engineer from a senior automation architect.

```
// ✅ 1. MEANINGFUL NAMES:
// ❌ Bad:  test('t1', ...)
// ✅ Good: test('user can login with valid credentials', ...)

// ✅ 2. ONE TEST = ONE JOB:
// ❌ Bad: 'login and search and checkout' (has "and" = split it!)
```

```
        // ✅ Good: separate tests for each behavior

        // ✅ 3. NEVER HARD-CODE VALUES:
        // ❌ Bad:  await page.goto('https://qa.myapp.com');
        // ✅ Good: await page.goto(config.baseUrl);

        // ✅ 4. ALWAYS USE POM:
        // ❌ Bad:  await page.locator('#email').fill('user@test.com');
        // ✅ Good: await loginPage.loginAs(config.username, config.password);

        // ✅ 5. CLEAR ASSERTIONS:
        // ❌ Bad:  await expect(page.locator('.msg')).toBeVisible();
        // ✅ Good: await expect(page.locator('.success-message'))
        //              .toHaveText('Login successful!', { timeout: 5000 });

        // ✅ 6. NO RANDOM WAITS:
        // ❌ Bad:  await page.waitForTimeout(3000); // Why 3? Nobody knows!
        // ✅ Good: await page.waitForSelector('.dashboard-title');

        // ✅ 7. INDEPENDENT TESTS:
        // ❌ Bad:  Test 2 needs Test 1 to have run first
        // ✅ Good: Every test sets up its own state via fixtures/beforeEach
```

🎤 **Interview Prep:** - *"What makes a test flaky and how do you fix it?"* → Flaky tests usually have hard-coded timeouts (`waitForTimeout`), shared state between tests, or poor locators that match multiple elements. Fix by using smart waits, making tests independent, and improving locators. - *"How do you approach code review for automation?"* → Check: meaningful test names, POM usage, no hard-coded values, no random waits, independent tests, clear assertions with failure messages, and `.env` used for credentials.

🎤 TOP 10 INTERVIEW POWER PHRASES

Question	Your Answer Keywords
Why Playwright?	WebSocket, auto-waiting, cross-browser, fast, TypeScript native
POM benefit?	Single place to update locators, separation of concerns, maintainability

How handle flaky tests?	Smart waits, independent tests, retry config, trace for debugging
CI/CD setup?	GitHub Actions, `npm ci`, `--with-deps`, GitHub Secrets, artifacts
Parallel execution?	Workers, isolated contexts, `test.describe.parallel`, `--workers` flag
API vs UI testing?	API: 10x faster, no browser, tests business logic; UI: tests user flow
Data-driven approach?	Array of objects, `for...of` loop, dynamic test names, one template
Environment management?	`process.env.ENV`, `.env` file, GitHub Secrets, never hard-code
Assertions best practice?	`toHaveText` vs `toContainText`, soft assertions, timeout param, `.not`
Framework design?	POM + Fixtures + Config + DDT + CI/CD + Reporting = scalable framework

🔥 QUICK COMMAND REFERENCE

```
# Run all tests:
npx playwright test

# Run specific file:
npx playwright test tests/login.spec.ts

# Run with specific browser:
npx playwright test --project=chromium

# Run with browser visible:
npx playwright test --headed

# Run by tag:
npx playwright test --grep @smoke
```

```
# Run specific environment:
$env:ENV="prod"  →  npx playwright test    # Windows
ENV=prod npx playwright test               # Mac/Linux

# Debug mode (pause on failure):
npx playwright test --debug

# View HTML report:
npx playwright show-report

# Update visual snapshots:
npx playwright test --update-snapshots

# Run parallel with N workers:
npx playwright test --workers=4

# Sequential (for debugging):
npx playwright test --workers=1
```

🎓 Built from Abiram's 2-month Playwright + TypeScript journey | Lessons 1–35 Complete ⭐ Repo: github.com/abiramprasad/playwright-automation